

FusionPRIME 架构设计

FusionPRIME: Fusion Plasma Reactive Integrated Modeling Ecosystem

文档状态：实现前架构提案，修订于 2026-08-04。

FusionPRIME 是面向聚变等离子体集成建模的 Python 平台。它将 veqpy、vtspy、mcdpy 及后续物理过程组织为可求解、可微分、可追溯的集成模型。

本文只固定职责边界和执行语义，不承诺最终类名、函数签名或存储实现。“必须”表示架构不变量，“应当”表示当前基线，“可以”表示不改变核心语义的实现选择。

- TODO:
- 1. WOrkflow 支持指定默认的历史记录开启和关闭, 如果关闭则 state 物化也全部是 0分配的

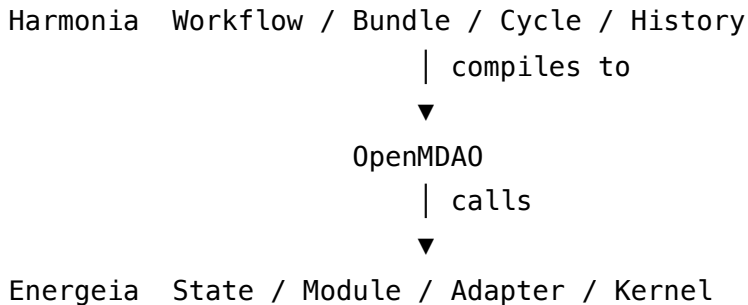
1. 架构概览

FusionPRIME 有三个领域层：

层	职责	主要概念
Energeia	定义物理数据和单个物理过程	State、Module、Adapter、Kernel、Record/Result
Harmonia	连接、求解、验收、求导并记录多个过程	Workflow、Bundle、Cycle、History
Theoria	可视化与交互呈现	物化路线：State/History；模块流程：Module/Workflow；具体 API 后定

Record 、 Result 、 RunContext 是 Energeia 定义、Harmonia 使用的不可变执行值；它们不得反向引用 Workflow、History 或 OpenMDAO 运行对象。

Theoria materialized views: State / History
 model-flow views: Module / Workflow
 concrete API: not designed yet



OpenMDAO 不是第四个领域层，而是 Harmonia 唯一的多 Module 执行和总导数底座。Harmonia 不实现第二套 scheduler、Picard/Newton 循环或全局导数传播器。

第一阶段范围

- 单机、单进程、共享内存；
- 顶层 Workflow 是顺序确定的 Module | Bundle | Cycle 线性序列；
- Bundle 表达同一入口状态上的语义并行；
- Cycle 表达局部反馈闭环；
- 拓扑和主要 shape 在 OpenMDAO setup 前确定；
- History 是内存中的线性 append-only 日志。

第一阶段不提供分布式调度、运行时动态图、History branch/fork/checkout/restart、任意 Python 自动微分、import 副作用式 Registry，也不自动把任意 State 映射为 IMAS/OMAS。

现有 veqpy、vtspy 和 mcdpy 仍混合状态、策略、fallback、workspace 与数值核心。第一阶段允许它们整体作为 Module 内部实现接入；边界经真实 Workflow 验证后，再逐步下沉为固定 Kernel ABI。

2. 执行与发布

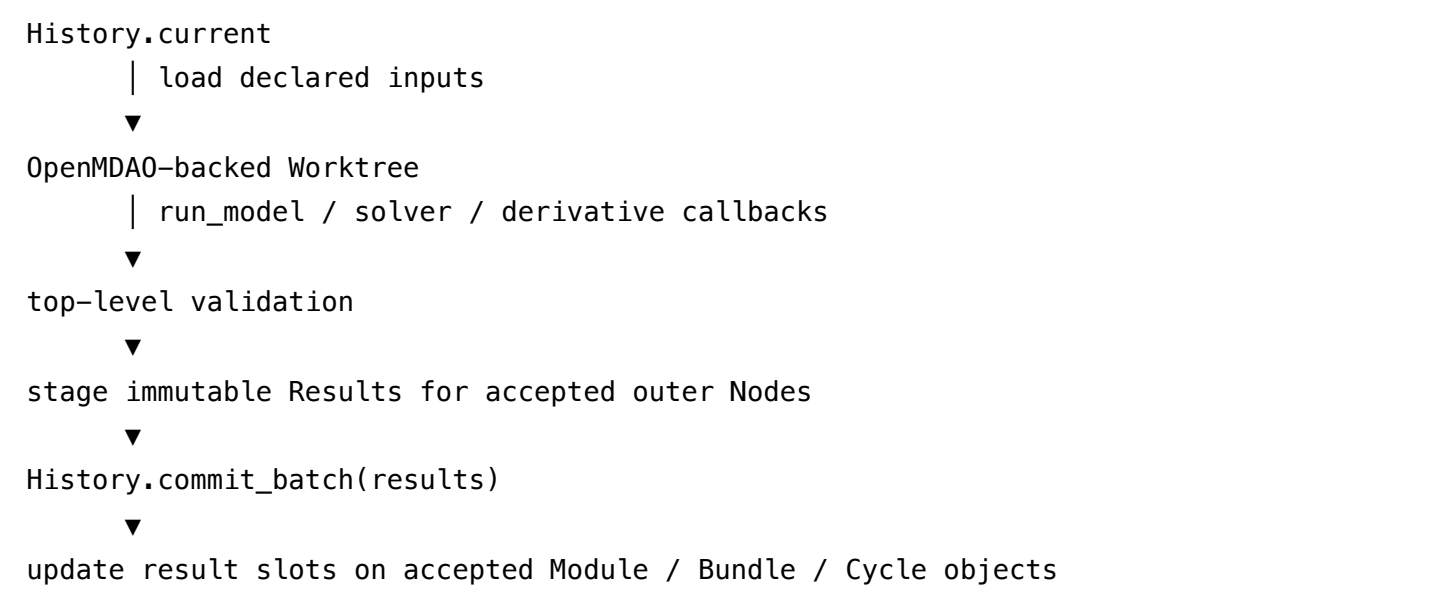
系统只区分两个生命周期平面：

平面	内容	性质
执行草稿	Workflow 的 Worktree；独立 Module 的 LocalStateBuffer	一次执行期间可覆盖

平面	内容	性质
已发布事实	State、Record、Result、Commit	发布后不可变

Worktree 是一次 OpenMDAO working state 的架构称呼；Vector 是其连续数值存储。KernelWorkspace 是 Kernel 私有缓冲，LocalStateBuffer 是独立调用的局部草稿。StateReader/StateWriter 只是对当前草稿的短生命周期能力视图，不拥有第二份 State。

Workflow 执行



关键规则：

1. evaluate() 可以因 Cycle、FD、CS 或线性化被多次调用，但只读写草稿并返回临时诊断。
2. 只有整个顶层 primal run 通过验收后，才物化新的 State 实例、构造 Result 并提交 History。
3. History 是 Workflow 的一致性权威；外层 Module/Bundle/Cycle 的 result 只是最近结果缓存。正常返回后，两者引用同一个 Result。
4. 每个成功外层 Node 产生一个 Result 和 Commit；内部 Module、Bundle 和迭代不产生 Commit。

每个 Module、Bundle 和 Cycle 运行对象各自拥有且只拥有一个最近 Result 槽；Node 只是外层角色，NodeBinding 和 Workflow 都不另存一份最近缓存。该槽对用户只读，由 owner token 保护的内部接受操作更新：独立 Module.solve() 调用自身接受操作，Workflow 在提交成功后调用外层对象的接受操作。公开 API 不允许任意注入 Result。

先提交 History、再更新这些槽，可避免缓存指向尚未进入事实日志的 Result。若缓存更新被异步中断，History 仍一致，执行实例进入 poisoned 状态并在重建时从已提交 batch 恢复缓存。

失败运行

后续 Node 失败时，本次 run 的成功前缀不提交，也不物化为公开 Result：

- 可归因到外层对象：提交一个 FAILED Result，StateVector 保持 run 起点不变，并只更新该 Module/Bundle/Cycle 的缓存；
- 无法归因到外层对象：提交 WORKFLOW 失败 Result，不更新任何运行对象缓存；
- 无法安全构造失败事实：抛出框架异常，不伪造 Commit。

失败后的 Worktree、solver 或 Kernel runtime 可能已被部分修改。除非能证明所有影响结果的状态都可由显式输入恢复，否则必须重建执行实例，不实现通用 rollback。

独立 Module

独立调用不创建一节点 Workflow，也不导入 OpenMDAO：

```
State inputs
  ▼
LocalStateBuffer
  ▼
evaluate → validate → materialize new State values → build Result
  ▼
module.result = Result
```

独立执行和 Workflow 执行必须复用同一物理 evaluate()、验证规则、诊断转换和 State 物化规则；区别只在草稿存储和事务边界。

3. State 与执行值

State

State 是具有独立物理意义、单位、shape 和不变量的数据边界，例如 EquilibriumState、CoreProfilesState 和 HeatingSourceState。Module 可以读取或产生多个 State；系统不要求一个巨型全局对象，也不由 IMAS/OMAS 路径反向决定物理划分。

具体 State 类型本身就是不可变 API 值，使用 frozen=True, slots=True 的 dataclass 定义：

```
@dataclass(frozen=True, slots=True, eq=True)
class EquilibriumState:
    snapshot_id: SnapshotId
    psi: FrozenArray = field(compare=False, hash=False)
    pressure: FrozenArray = field(compare=False, hash=False)
    schema_version: str = field(compare=False)
```

执行草稿从来不是 State 实例；物化是从 Worktree 或 LocalStateBuffer 的声明字段直接构造一个新的 State 实例。State 本身已经不可变，因此不再引入 StateSnapshot 这一平行类型名。

- StatePath：根 State 的稳定逻辑名称；
- FieldPath：State 内可存储叶字段的相对路径；
- StateSpec：字段、dtype、shape、单位、连续性和领域不变量的 schema。

完整字段身份是 (StatePath, FieldPath)。

被发布的 State 实例不得引用会被草稿或 workspace 覆写的内存，不得暴露公共可写 owner/alias，嵌套容器也必须不可变。未变化的数组块和子结构可以安全共享。

具体 backing 可以是复制后的只读块、不可变 buffer 或分块存储，由原型决定；架构只要求无法通过公开引用改写已发布值。writeable=False 或 frozen=True 本身不是完整保证。

StateSpec 或其关联 codec 负责把 State 实例装入草稿，以及从声明字段物化新的完整 State 实例。这样 Module、Bundle 和 Cycle 不需要各自实现一套 State 拼装逻辑。

State 内可以提供派生量惰性缓存。缓存位于快照外部，可驱逐，不进入 Commit，也不得成为物理正确性的来源。

dataclass 相等性

frozen=True / slots=True 与 eq=True 是不同问题。所有公共 API 数据值都应当是 frozen/slots dataclass，但是否使用值相等由类型语义决定，不能由 pytest 的便利性决定。

当前能确认的生产用途有两类：

- StatePath、FieldPath、schema/config、StateVector 等小型值，需要精确相等来做合同校验、映射查找和编译缓存键；它们适合 eq=True；
- State、Record、Result、Commit 等带稳定身份的已发布事实使用 eq=True 时，应只比较稳定 ID，使序列化前后的同一事实仍可识别，也可安全用于映射查找；不得递归比较大型数组或诊断 payload。

State 的数值等价、近似相等和存储去重是三个不同操作：数值等价应显式指定字段、单位、容差与 NaN 规则；去重应使用明确的内容摘要；Workflow 和 History 不通过 State.__eq__ 判断是否跳过执行或产生新版本。若 State 保留 dataclass 生成的 eq=True，数组字段必须 compare=False，相等性由 snapshot_id 等稳定身份字段决定；若最终没有身份比较的生产需求，则应改为 eq=False。该选择由原型验证，不让测试代码反向决定 API。

Record、Result 与 KernelResult

值	含义
KernelResult	一次 Kernel 调用的数值输出和原始诊断
Record	一次已发布逻辑执行的不可变事实
Result	Record 加本次实际输出的 State

Record 至少保存 run/执行对象标识、类型、status、message、计时、计数、Kernel/fallback 路径、错误摘要、provenance 和可选 children。类型覆盖 INIT、MODULE、BUNDLE、CYCLE、WORKFLOW 及内部诊断，但内部诊断不对应 Result 或 Commit。status 是成功与否的唯一权威；无法从稳定 API 获得的计数为 None，不得猜测。

RunContext 只携带 run_id、运行模式和 provenance，不保存 History 游标、调度位置或“是否位于 Cycle”一类控制开关。

```
@dataclass(frozen=True, slots=True, eq=True)
class Result:
    result_id: ResultId
    record: Record = field(compare=False, hash=False)
    outputs: FrozenMap[StatePath, State] | None = field(
        compare=False,
        hash=False,
    )
```

这里的 eq=True 表示事实身份相等：序列化前后的两个 Result 只要 result_id 相同即可匹配；两个不同 run 即使数值输出完全相同也不是同一个 Result。Record、Commit 和 State 采用同样原则时，稳定 ID 必须由构造入口保证唯一且可序列化。

成功 Result 只包含该外层边界实际产生的完整输出 State，不包含全局 StateMap、Commit 或 History。失败 Result 的 outputs 为 None，不得把继承状态伪装成本次输出。

Result 通过引用重绑定更新，不原位修改：旧 Commit 始终保留旧 Result，最近缓存指向新 Result。最近缓存不是同一 run 的一致视图；需要一致视图时查询 History 的 run batch。

4. Energeia：单个物理过程

Module

Module 是用户可独立理解、配置、运行和诊断的完整物理过程，例如 VEQ、VTS 或 MCD。默认粒度是物理过程，而不是每个小型数值函数。

Module 必须声明 State/Field 端口、单位、shape 和连续性；持有物理策略与不可变配置；通过 Adapter 调用 Kernel；实现验收、fallback 和诊断解释；声明局部导数能力；支持不依赖 Harmonia 的独立 solve()。

Module 不得调度其他 Module，不得读取未声明字段，不得依赖 OpenMDAO name、Vector、Problem、History 或 Commit，也不得保存跨调用 reader/writer 或可写输入 alias。

```
class Module(Protocol):
    ports: Ports
    derivatives: DerivativeCapabilities
    result: Result | None

    def evaluate(self, inputs, outputs, context) -> DiagnosticDraft: ...
    def solve(self, initial: Mapping[StatePath, State]) -> Result: ...
```

evaluate() 是唯一物理计算入口，只操作借用草稿并返回 alias-free 的 DiagnosticDraft。Module 负责物理验证和把诊断解释为 Record；StateSpec/codecs 负责物化新的 State 实例；共享收尾逻辑组装 Result。独立 solve() 与 Workflow 复用同一收尾逻辑。

最近结果槽属于 Module 自身，但更新时间由拥有执行事务的入口决定。公开 API 不提供任意 publish(result)；内部接受操作必须验证 owner token 和 Result 归属，避免未验收 Result 被注入 Module。

多 Kernel 与 fallback

Module 与 Kernel 不要求一一对应。Module 可以先运行 Powell Kernel，验收失败后再运行 LM Kernel；选择策略、fallback 原因和成功判定属于 Module，Kernel 只实现固定数值合同。

Module 位于 Bundle 或 Cycle 内部时只执行 `evaluate()`：不物化公开 State 实例，不构造独立 Result/Commit，不更新自身 `module.result`；临时诊断由父对象转换为 `Record.children`。若该 Module 在绑定前曾独立运行，旧 `module.result` 可以保留，但它不代表父 Bundle/Cycle 的当前执行；其 `run_id/provenance` 必须能够区分来源。

Adapter

Adapter 是 State API 与 Kernel ABI 的唯一翻译层。它负责提取物理输入、转换单位/坐标/dtype/layout/模式码、验证 topology、写回 KernelResult，以及映射 tangent/cotangent。

Adapter 不持有 History、Result、Problem、Vector，也不跨调用保存 reader/writer。不同物理过程可以使用手写 Adapter；系统不假设万能转换器。

Kernel ABI

Kernel 是不感知 State、Module、Workflow 和 History 的高性能数值核心：

类型	内容
KernelTopology	静态规模、布局、索引、残差维度和实现族
KernelInstance	本次物理输入、边界、源项和显式初值
KernelConfig	非物理求解控制和已数值化模式
KernelResult	数值输出、状态码、收敛信息和小型诊断

Kernel 可以保留 topology 相关 workspace、编译产物和只影响性能的缓存。任何可能改变解支、fallback、收敛成败或成功判定的 warm start，都必须进入显式 State、KernelInstance 或 KernelConfig。

Kernel 可选提供 residual、Jacobian、JVP、VJP 和局部线性求解；它不负责跨 Module 链式法则或 Cycle 全局耦合系统。

实例身份

配置、StateSpec 和 KernelTopology 可以共享，但带 workspace、warm start 和最近结果槽的运行实例不能共享执行位置。

同一 Module/Bundle/Cycle 实例在成功 setup 后永久绑定到一个位置。实例只保存不透明 owner token；稳定 `node_id`、system path 和版本映射由 Harmonia 的内部 `NodeBinding` 持有。`NodeBinding` 不保存最近 Result。setup 失败必须释放全部预留。

绑定后的 Module 不再允许独立 `solve()`，以免复用 runtime 破坏 workspace、warm start 和结果身份。

5. Harmonia：组合与求解

Workflow 与 Node

Workflow 是线性外层序列、领域编译器、OpenMDAO facade 和顶层验收边界：

```
workflow = Workflow(  
    VEQModule(...),  
    Bundle(HeatingModule(...), CurrentDriveModule(...)),  
    Cycle(VTSMModule(...), VEQModule(...), solver=Picard(...)),  
    MCDModule(...),  
)
```

Node 是 Module、Bundle 或 Cycle 在 Workflow 外层扮演的角色，不是新的用户对象。外层 Module 的最近结果仍在 `module.result`，外层 Bundle/Cycle 分别使用 `bundle.result` / `cycle.result`；不存在 `Node.result`、Node wrapper 缓存或 Workflow 侧副本。

编译器为每个位置生成内部 `NodeBinding`，保存稳定身份、system path、端口、版本映射和物化清单，但不拥有最近结果。用户对象不需要知道 Commit、History 或 OpenMDAO 绝对变量名。Module 不继承 OpenMDAO Component，编译器通过组合 wrapper 连接两者。

Workflow compiler 必须：

1. 预留并单绑定全部实例；
2. 校验读写集、单位、shape、KernelTopology 和唯一生产者；
3. 编译字段连接、Bundle 和 Cycle；
4. 生成 StateCatalog、版本清单和逐 Node 物化清单；
5. 建立唯一 OpenMDAO Problem。

Harmonia 可以暴露只读诊断和图视图，但不得提供允许外部直接 `set_val()`、`run_model()` 或改变 working state 的 Problem 逃生口。

Worktree 与字段版本

Module 不获得整棵全局状态，只获得声明端口对应的能力。输入只读，输出只能写声明字段，reader/writer 不得跨越当前 callback。Kernel 若需原位修改输入，Adapter 必须复制到独立 workspace。

OpenMDAO 要求每个 output 只有一个 source，而 Module 可能只更新 State 的部分字段。Harmonia 因此使用编译期字段级 SSA：

```
profiles@0 = {temperature: T@0, density: N@0}
Module A writes temperature
profiles@1 = {temperature: T@1, density: N@0}
Module B writes density
profiles@2 = {temperature: T@1, density: N@1}
```

FieldVersion 是 (StatePath, FieldPath, generation)；StateRevision 是某位置完整的字段版本映射。generation 表示静态图位置，不表示运行次数。

编译器保存 FieldVersion 到 OpenMDAO output 的版本清单，以及每个外层 Node 的完整输出 revision、诊断和 provenance 物化清单。run_model() 返回后，Workflow 通过公共读取 API 物化新的输出 State 实例；提交时不复制整个 Vector，未变化 State 由 History 复用旧版本。

DiagnosticDraft 不能采用“最后一次 callback 获胜”。NodeBinding 只选择与本次已接受 primal 状态对应的诊断：Cycle 中间迭代可以进入父 Record.children，FD/CS、coloring 和导数模式的 draft 必须隔离且永不进入正式 Result。实现可以把诊断映射为显式 callback 输出，或使用带 run_id/mode/call token 的 staging，但选择规则必须一致。

Bundle

Bundle 表示多个子 Module 对同一入口 revision 的独立求值。所有子 Module 都看不到同批兄弟的输出。第一阶段在 StatePath 粒度上要求写集互斥；是否放宽到可证明安全的 FieldPath 粒度由原型决定。

Bundle 可以编译为普通 Group 或 ParallelGroup，但领域结果不得依赖实际调度方式。外层 Bundle 只产生一个 Result/Commit，子诊断进入 Bundle Record.children；Bundle 嵌入 Cycle 时不单独发布。

Bundle 不是一般控制流。条件分支留在 Module 的显式配置和 fallback 中；未来若需要结构化分支，应另设组合子。

Cycle

Cycle 表示多个 Module（以及可选的内部 Bundle）之间的自洽条件，编译为 OpenMDAO Group，并由 OpenMDAO nonlinear/linear solver 负责迭代和隐式导数。

Cycle 只有一个外层 Result/Commit：内部迭代、Module 和 Bundle 不发布 Result；收敛后的边界 revision 物化为新的输出 State 实例；内部失败归因到外层 Cycle，具体位置进入 error 或 children；非收敛必须作为顶层失败传播。

残差历史和迭代 telemetry 不是权威 State。它们在每次 primal run 前重置，在 FD、CS、coloring 和总导数期间禁用或隔离，并允许限长、采样或外置。

Workflow API

```
class Workflow:
    def setup(self) -> None: ...
    def create_history(self, initial) -> History: ...
    def run(self, history: History) -> tuple[Commit, ...]: ...
    def compute_totals(self, *, of, wrt, method=None): ...
    def graph(self) -> ReadOnlyGraph: ...
```

run() 校验 catalog fingerprint，装入 History.current，调用一个 OpenMDAO Problem，完成验收、Result staging、History batch commit 和缓存更新。Workflow 不自己遍历 Node 执行物理代码。

第一阶段不允许同一 Workflow/Node 并发或重入，也不保证发布窗口中的跨线程读取一致性。

6. History

History 是线性事实日志，不是运行时，也不是树：

```
Commit sequence
+
StatePath E: [E0, E1, ...]
StatePath P: [P0, P1, ...]
StatePath Q: [Q0, Q1, ...]
```

Workflow 在 setup 时生成 StateCatalog，固定 StatePath、StateSpec 和槽位顺序。每个 Commit 强引用同一个 Result，并保存不可变 StateVector；其中每个整数选择相应版本列中的

State。StateMap 是由 catalog、版本列和 StateVector 解析出的只读完整状态视图，不是另一份持久化字典。

```
class History(Sequence[Commit]):
    @property
    def head(self) -> Commit: ...
    @property
    def current(self) -> StateMap: ...

    def commit(self, result: Result) -> Commit: ...
    def commit_batch(self, results) -> tuple[Commit, ...]: ...
    def states(self, commit) -> StateMap: ...
    def run(self, run_id) -> tuple[Commit, ...]: ...
```

创建 History 时，每个 slot 必须提供合法初始 State，并生成一个 INIT Commit。

提交规则：

- History.commit() 只接收 Result；
- 成功 outputs 追加到对应版本列，其余 StateVector 下标沿用 HEAD；
- 失败 Result 不追加 State，复用 run 起点 StateVector；
- commit_batch() 先完整验证并在 staging 中推演，再一次公开整个 batch；
- Commit 不保存 parent，序列位置是唯一顺序权威。

History 不提供 branch、fork、merge、checkout 或任意旧 Commit restart。不同参数扫描、优化 evaluation 或物理场景使用不同 History；旧 Commit 可读取，但不是运行游标。

7. 导数、Reactive 与失败隔离

导数责任分为三层：Kernel 提供局部 residual/Jacobian/JVP/VJP/线性求解；Adapter 和编译生成的 DerivativeBinding 映射数组布局；OpenMDAO 负责跨 Module 传播、Bundle 独立块、Cycle 耦合系统、relevance、coloring 和总导数。

导数热路径不得构造 State、Record、Result、Commit 或 StateMap。FD、CS、coloring discovery、linearization 和 compute_totals 也不得更新最近缓存、History 或影响下一次 primal 的 warm start。

对 $z = G(z, x)$ ，Cycle 在收敛点求隐式灵敏度：

$$(I - G_z) dz = G_x dx$$

不反向展开 Picard 迭代。未收敛 Cycle 没有有效总导数。

含分支、fallback 或多解跳支的 Module 可以使用 FD/CS，但诊断必须说明活动路径是否变化；框架不得把这类差分无条件称为精确导数。

FusionPRIME 只保留 State 内部的派生量惰性缓存，不实现 primal Workflow memoization、输入版本比较或自研失效图。OpenMDAO relevance/coloring 是导数优化，不是 primal reactive scheduler。

物理/数值失败形成规范 FAILED Result；合同违反、资源耗尽或无法安全发布事实属于框架失败，直接抛出异常。失败或异步中断后，若内部状态无法证明可恢复，则重建 Problem、wrapper 和相关 Kernel runtime，不遍历任意 Python 对象回滚隐藏状态。

8. 架构不变量

1. Module 只通过声明端口访问 State，不依赖 OpenMDAO 或 History。
2. Adapter 是 State 与 Kernel ABI 的唯一翻译层；Kernel 不感知领域对象。
3. evaluate() 只修改执行草稿，不发布事实。
4. 每次已接受的外层执行都新建 Result，不修改旧对象。
5. Bundle 和 Cycle 各只有一个外层 Result/Commit；内部求值只进入父诊断。
6. OpenMDAO 是唯一多 Module 执行、Cycle 求解和全局导数引擎。
7. History 只接收 Result，并以线性 Commit 加版本列保存状态。
8. 失败 run 不提交成功前缀，也不发布半成品 State。
9. 导数探针不修改 Result 缓存、History 或物理 warm start。
10. 同一运行实例只绑定一个执行位置；第一阶段不并发、不重入。
11. 所有公共 API 数据值，包括 StatePath/FieldPath/StateSpec、Ports、Module Config、Kernel ABI 值、State、RunContext/DiagnosticDraft、Record/Result、FieldVersion/StateRevision、StateCatalog/StateVector、Commit、manifest 和 derivative binding，都必须使用 frozen=True, slots=True 的 dataclass。
12. 这些值的公开嵌套容器和数组必须深度不可变；dataclass 的 frozen=True 不足以保护可写数组 alias。
13. Module、Bundle、Cycle、Workflow、History 和 Kernel runtime 可以持有可变资源，但只能发布不可变值。
14. Module/Bundle/Cycle 各自持有一个最近 Result 槽；Node、NodeBinding 和 Workflow 不持有第二份最近缓存。

- 15. eq=True 只表达小型值相等或稳定事实身份，不承担 State 数值近似、Workflow 失效判断或 History 去重。
- 16. 正式 Result 只使用已接受 primal 状态对应的诊断；Cycle 中间迭代与导数探针不得以“最后调用”覆盖正式诊断。

9. 工程依据与路线

真实 VEQ → MCD 原型使用 OpenMDAO 3.45.0，输出与直接调用逐元素一致：

路径	中位时间	相对直接调用
直接 VEQ → MCD	19.360 ms	-
OpenMDAO, 17 个字段变量	19.426 ms	+0.067 ms
OpenMDAO, 打包 State	19.394 ms	+0.035 ms

这说明 OpenMDAO 对完整物理 Module 的附加代价较小，但对约 0.1 ms 的微型 Kernel 可能与 Kernel 本身同量级。因此默认 Component 粒度应是 Module；更值得测量的是 FieldVersion 常驻内存、workspace、State 物化和 History 长期成本。

原型还表明 coloring 可以显著减少导数扰动次数，而普通 run_model() 仍执行完整 primal 图。这支持“使用 OpenMDAO 导数优化，不自研 primal reactive scheduler”的边界。

实现顺序

1. **值对象与 History**：固定 frozen/slots dataclass 合同、数组深度不可变性、State/Record/Result、StateCatalog/Vector/Commit、版本列、StateMap 和原子 commit_batch()。
2. **Module 双入口与编译绑定**：实现 LocalStateBuffer、reader/writer、统一收尾、受 owner token 保护的最近结果接受操作、wrapper、FieldVersion/Revision、NodeBinding 和物化清单。
3. **真实物理链**：接入 VEQ、VTS、MCD，修正 State 粒度、失败语义和内存模型，验证独立 solve 与 Workflow 一致。
4. **Bundle、Cycle 与导数**：实现组合语义、OpenMDAO solver、children、DerivativeBinding、FD/CS、coloring 和隐式总导数。
5. **Kernel 纯化与规模化**：固定各包 Kernel ABI；profiling 后再决定零拷贝、chunk、压缩、持久化、共享内存和 MPI。

必须由原型回答

- VEQ/VTs/MCD 的最终 State 与 Field 粒度；
- State 数组 backing、分块、去重和持久化策略；
- SnapshotId/ResultId 等稳定事实 ID 的生成、序列化及 eq/hash 语义；
- FieldVersion 数量、Vector 常驻内存与物化峰值；
- Explicit/Implicit Module 的最小统一导数协议；
- Bundle 写冲突是否可安全放宽到 FieldPath；
- Cycle telemetry 的完整性、限长和探针隔离；
- failure 后哪些 runtime 可 reset，哪些必须 rebuild；
- History 已提交而缓存更新中断时的恢复机制。

10. 最简摘要

Energieia:

State → Module → Adapter → Kernel

Harmonia:

Workflow[Module | Bundle | Cycle]

→ OpenMDAO-backed Worktree

→ validate and stage Result

→ History.commit_batch(Result)

→ update result slots on accepted Module / Bundle / Cycle

Standalone Module:

LocalStateBuffer → same evaluate/materialization rules → Result

核心边界只有五条：

1. State 是物理语言；Worktree 和 LocalStateBuffer 只是可覆盖草稿。
2. Module 定义物理过程，Adapter 翻译，Kernel 只做固定数值计算。
3. Workflow 线性组织外层 Node；Bundle 表达同入口并行，Cycle 表达反馈自洽。
4. 只有验收后的外层执行才产生 Result；History 是 Workflow 的一致性权威。
5. Kernel 负责局部导数，OpenMDAO 负责全局传播，导数探针不发布事实。